

ChatGPT ▾

🔗 Dela

👤 Lägg till personer

...

Example of handling an HTTP request using Jetty

show an example of doing an HTTP request using Jetty.

Suggest a bug free, up to date Java library for making and receiving HTTP calls.

Show an example of both receiving and sending with OkHttp.

Show an example of doing REST calls and servicing REST calls with NanoHTTPD

Can you give me the maven coordinate for OkHttp.

Jag letar efter en privat och enkel chat grupp, har du några förslag ?

Går det att sätta upp en privat chat mellan 10 användare med hjälp av Craft ?

Enkelt, anonymt chatforum

Detta svar var uppåt väggarna total fel! Är det för att jag skrev på engelska ?

I Java så finns det http server och klient under paketet `com.sun.net`. Finns dom kvar i senare versioner av Java, som till exempel Java 12 eller senaste versionen av java ?

Ja!

Ett REST-liknande mini-ramverk i groovy.

Please replace JsonSlurper with something else since
JsonSlurper has known bugs in it.

Absolutely! If you want to **avoid** `JsonSlurper` due to its quirks, a solid alternative is **Gson** (`com.google.gson`) or **Jackson**. Both are mature, widely used, and reliable for JSON parsing in Java/Groovy.

Here's how we can modify your **mini REST framework in Groovy** to use **Gson** instead of `JsonSlurper` .

✅ Updated Example Using Gson

Maven dependency for Gson (if using Maven/Gradle):

xml

 Kopiera kod

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.10.1</version>
</dependency>
```

App.groovy (REST server)

groovy

 Kopiera kod

```
import com.sun.net.httpserver.HttpServer
import com.google.gson.Gson

def gson = new Gson()
def router = new Router()

// --- REST ENDPOINTS ---

router.get("/hello") { params, body ->
    [message: "Hello from Groovy REST mini-framework with Gson"]
}
```

```
router.get("/users/:id") { params, body ->
    [userId: params.id, status: "found"]
}

router.post("/echo") { params, body ->
    def parsed = gson.fromJson(body, Map)
    [youSent: parsed]
}

router.put("/items/:id") { params, body ->
    def parsed = gson.fromJson(body, Map)
    [updated: params.id, newData: parsed]
}

router.delete("/items/:id") { params, body ->
    [deleted: params.id]
}

// --- START SERVER ---

def server = HttpServer.create(new InetSocketAddress(8080), 0)
server.createContext("/", { exchange -> router.handle(exchange) } as com
server.executor = null
server.start()

println "Server running at http://localhost:8080"
```

Router.groovy

No changes needed here; it still converts responses to JSON via

`JsonOutput.toJson()` — you could optionally switch that to `Gson` as well for consistency:

groovy

 Kopiera kod

```
import com.sun.net.httpserver.HttpExchange
import com.google.gson.Gson

class Router {
    private routes = []
    private gson = new Gson()

    void get(String path, Closure handler) { route("GET", path, handler)
    void post(String path, Closure handler) { route("POST", path, handler)
    void put(String path, Closure handler) { route("PUT", path, handler)
    void delete(String path, Closure handler) { route("DELETE", path, handler)

    private void route(method, path, handler) {
        routes << [method: method, path: path, handler: handler]
    }

    void handle(HttpExchange exchange) {
        def reqMethod = exchange.requestMethod
        def reqPath = exchange.requestURI.path

        for (r in routes) {
            def params = matchPath(r.path, reqPath)
            if (r.method == reqMethod && params != null) {
                def body = exchange.requestBody.text
                def result = r.handler.call(params, body)

                def json = gson.toJson(result)
                exchange.responseHeaders.add("Content-Type", "application/json")
                exchange.sendResponseHeaders(200, json.bytes.length)
                exchange.responseBody.withStream { it.write(json.bytes) }
                return
            }
        }

        // No match → 404
        exchange.sendResponseHeaders(404, 0)
        exchange.responseBody.close()
    }
}
```

```
}

private Map matchPath(String routePath, String realPath) {
    def routeParts = routePath.split("/")
    def realParts = realPath.split("/")

    if (routeParts.size() != realParts.size()) return null

    def params = [:]
    for (int i = 0; i < routeParts.size(); i++) {
        if (routeParts[i].startsWith(":")) {
            params[ routeParts[i].substring(1) ] = realParts[i]
        } else if (routeParts[i] != realParts[i]) {
            return null
        }
    }
    return params
}
}
```

Benefits of using Gson

- Fully tested, no known Groovy-specific bugs like `JsonSlurper` .
- Can parse JSON into `Map` / `List` or custom classes.
- Converts Groovy maps back to JSON safely.
- Very fast and actively maintained.

If you want, I can also **upgrade this mini-framework** so that **both JSON parsing and serialization fully use Gson**, making it fully consistent and safer for production use.

Do you want me to do that?

